

Apple Retail Sales Analysis Dashboard

Data Model Documentation (2020-2024)

Prepared by Jifu Jifrin

1.Tables Overview

1. Sales (Fact Table)

Key Columns:

- **Sale_date**
- **month**
- **product_id**
- **quantity**
- **Price of One Product**
- **Store_ID**

Purpose:

This is the **central fact table** and the core engine of the data model.

It stores all transactional sales records and is used to calculate key KPIs such as:

- Total Revenue
- Total Quantity Sold
- Total Orders
- Average Order Value

All dimension tables connect directly or indirectly to this table.

2.Products (Dimension Table)

Key Columns:

- **Product_ID**
- **Category_ID**
- **Price**
- **Launch_Date**
- **month**

Purpose:

This table contains **product-level descriptive information**.
It enables detailed analysis such as:

- Product-wise Revenue
- Top & Bottom Selling Products
- Product Contribution to Category Revenue

Connected to **Sales** via `product_id`.

3. Category (Dimension Table)

Key Columns:

- **category_id**
- **category_name**

Purpose:

This table defines **product categorization** and supports high-level aggregation.
It enables insights such as:

- Revenue by Category
- Category-wise Performance Comparison
- Drill-down from Category → Product

Connected to **Products** via `category_id`.

4. Stores (Dimension Table)

Key Columns:

- **Store_ID**
- **City**
- **Country**
- **Region**

Purpose:

This table contains **store and geographical attributes** used for location-based analysis, including:

- Store-wise Revenue
- Country-wise Sales
- Region-wise Performance Comparison

Connected to **Sales** via `Store_ID`.

5.Warranty (Dimension Table)

Key Columns:

- **claim_id**
- **claim_date**
- **month**
- **repair_status**

Purpose:

This table stores **warranty claim and repair information** for sold products.
It supports warranty-related KPIs such as:

- Total Warranty Claims
- Claim Rate
- Repair Status Distribution (Completed, Pending, Rejected)

Connected to **Sales** via `Sale_id`

2.Relationships Overview

From Table	Column	To Table	Column	Cardinality
Category	category_id	Products	Category_ID	One → Many
Products	Product_ID	Sales	product_id	One → Many
Stores	Store_ID	Sales	Store_ID	One → Many
Sales	sale_date / month	Warranty	month	One → Many

3.Key Calculations / Measures (Used Across Dashboards)

1.Sales Table

Measures:

average Order Value `DIVIDE(`

```
SUM(sales[Total]),  
DISTINCTCOUNT(Sales[sale_ID]))
```

Gauge Chart Creating (maximum,minimum,target)
Total Orders `DISTINCTCOUNT(sales[sale_id])`
Total Quantity `SUM(Sales[Quantity])`
Total Sales `SUM(sales[Total])`

Calculated Columns:

Price of One Product `RELATED(products[Price])`

2.Products Table

Measures:

Total Products `DISTINCTCOUNT(products[Product_Name])`

3.Stores Table

Measures:

Total Stores `COUNT(stores[Store_ID])`

Calculated Columns:

Region

4.Warranty Table

Measures:

Claim Rate% `DIVIDE([Total Claims], [Total Orders])`

Completed Repairs `CALCULATE(`

```
    COUNTROWS(warranty),  
    warranty[Repair_Status] = "Completed"  
)
```

In Progress Repairs `CALCULATE(`

```
    COUNTROWS(warranty),  
    warranty[Repair_Status] = "In Progress"  
)
```

Pending Repairs `CALCULATE(`

```
    COUNTROWS(warranty),  
    warranty[Repair_Status] = "Pending"  
)
```

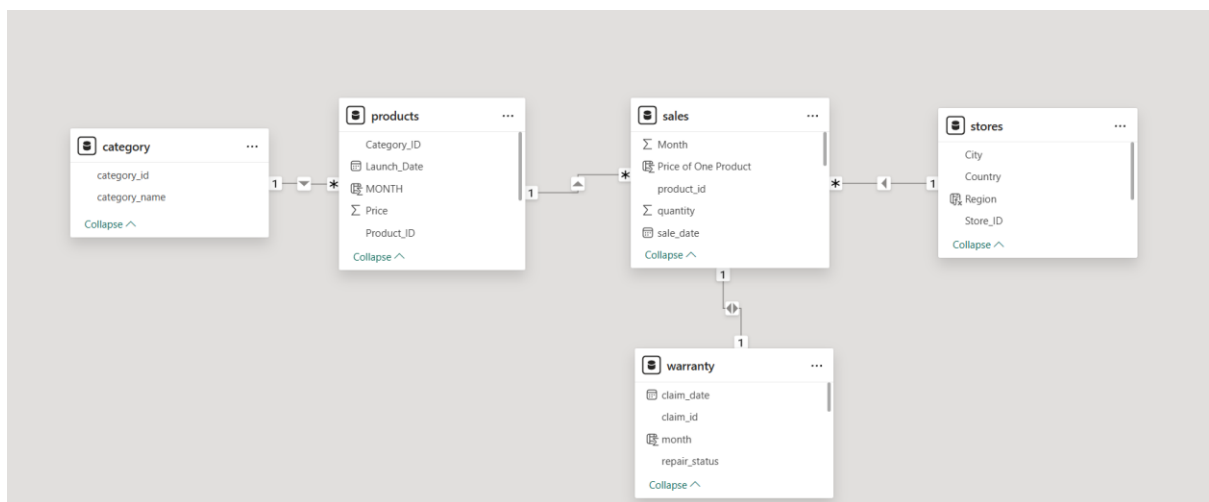
Rejected Repairs `CALCULATE(`

```
    COUNTROWS(warranty),  
    warranty[Repair_Status] = "Rejected"  
)
```

Total Claims `COUNT(warranty[sale_id])`

4. Notes & Assumptions

- Each sale is associated with **one product and one store**
- Category information is maintained separately to support clean hierarchy
- Warranty data is analysed at a transactional level
- Month column is used for time-based analysis instead of a separate Date table
- Model finalized on **Day 5** after validation and optimization



5. Data Model Design

- **Schema Type:** Snowflake Schema
- **Fact Table:** Sales
- **Dimension Tables:** Products, Warranty, Stores
- **Sub Dimension Tables :** Category

Design Advantages:

- Clear separation of fact and dimension data
- Reduced data redundancy through normalization
- Efficient aggregation and filtering
- Optimized performance in Power BI with well-defined relationships